EMBEDONE

EmbedOne IDE Handbook

Install it, pick a board, and build, flash, monitor and debug from one window.

VERSION	PLATFORM	PUBLISHED
1.0.0	Windows 10 and 11, x64	November 2026

Contents

1	What EmbedOne is	Start here
2	Installing EmbedOne	Download, verify, first launch
3	A tour of the window	Workspace Home and shortcuts
4	Your first project: an STM32 Nucleo	Build, flash, monitor, debug
5	ESP32 and Raspberry Pi Pico	Arduino-based boards
6	Linux boards: Raspberry Pi and NXP i.MX	Build and run over SSH
7	The serial console	
8	Adding more boards	Board discovery
9	The command line	Every command
10	The project file	`embedone.json`
11	Files, folders and settings	
12	Troubleshooting	
13	Updating and uninstalling	
14	Known limitations in 1.0.0	
A	Supported boards	All 33

1 What EmbedOne is

EmbedOne is a desktop IDE for embedded development. You choose a board; EmbedOne fetches the exact compiler, build tools and chip support files that board needs, and then lets you build, flash, watch serial output and debug from one window.

Embedded work usually means a different vendor IDE, flashing tool and serial monitor for each family of chip, plus an afternoon spent installing toolchains and fixing paths. EmbedOne replaces that with one workflow that looks the same whether the target is an STM32 microcontroller, an ESP32, a Raspberry Pi Pico, or a Linux board such as a Raspberry Pi 5 or an NXP i.MX 8.

What it does for you

Step	What EmbedOne does
Set up	**Doctor** checks what your board needs and **Install** downloads it: compilers, CMake, Ninja and the chip's support files. Every download is checksum-verified and kept in EmbedOne's own folder, so other tools on your PC are left alone.
Build	Compiles your project and shows how much of the chip's flash and RAM it uses.
Flash	Puts the firmware on the board. On STM32 boards it first reads the chip's identity and refuses to write if it isn't the chip your project targets, then reads the flash back to verify it.
Monitor	Opens a serial console you can read and type into.
Debug	Breakpoints, stepping, variables and the chip's peripheral registers, on STM32 boards.

Everything the app does is also available as the `embedone` command, so the same steps can be scripted from PowerShell or a build server (chapter 9).

What's in version 1.0.0

Family	Boards	Build	Flash	Monitor	Debug
STM32 Nucleo	3	Yes	Yes	Yes	Yes
ESP32 (Arduino)	1	Yes	Yes	Yes	No
Raspberry Pi Pico family (Arduino)	4	Yes	Yes	Yes	No
Raspberry Pi Linux boards	10	Yes, on the board	Deploys the app	Runs the app	No
NXP i.MX Linux boards	15	Yes, on the board	Deploys the app	Runs the app	No

You can also add more Arduino boards yourself (chapter 8). The full list is in appendix A.

2 Installing EmbedOne

What you need

- Windows 10 or 11, 64-bit.
- About 470 MB of disk space for the application.
- An internet connection and several GB of free space the first time you set up a board, for its compiler and support files. These are downloaded only for the boards you use.
- No administrator rights. EmbedOne installs for your user account only.

Install

1. **Download the installer**
   `EmbedOne-1.0.0-Setup-x64.exe` from the EmbedOne website or github.com/EmbedOne/embedone-web/releases.

2. **Check it (recommended)**
   In PowerShell, run `Get-FileHash .\EmbedOne-1.0.0-Setup-x64.exe` and compare the result with the SHA-256 published on the release page.

3. **Run it**
   The installer isn't code-signed yet, so Windows SmartScreen will say "*Windows protected your PC*". Choose **More info**, then **Run anyway**.

4. **Choose your options**
   EmbedOne installs into `%LOCALAPPDATA%\Programs\EmbedOne` and adds a Start menu shortcut. A desktop shortcut is optional. The installer also adds the `embedone` command to your user PATH.

First launch

Open **EmbedOne** from the Start menu. Three things may happen the first time, and all of them are expected:

> **Windows Firewall asks about "VSCodium"**
> EmbedOne is built on VSCodium, the open-source build of the VS Code editor, and some Windows dialogs still use that name. EmbedOne doesn't need incoming connections, so choosing **Cancel** is fine.

> **"Do you trust the authors of the files in this folder?"**
> The editor asks this the first time you open a workspace. Trust folders you created yourself; EmbedOne needs trust to run builds.

Workspace Home opens

This is EmbedOne's dashboard, described in the next chapter. A short Getting Started walkthrough is also available from it.

3 A tour of the window

EmbedOne opens on **Workspace Home**, a dashboard rather than a sidebar full of panels. It lists your recent workspaces, the projects in the current one, and every board you can target.

Workspaces and projects

A **project** is one program for one board: a folder with an `embedone.json` file and your source code. A **workspace** groups several projects, for example the firmware for a sensor board and the Linux application that reads from it. Each project builds on its own; the workspace just keeps them together.

The title bar

While you edit, the title bar keeps the controls you use most: **Home**, **Open file**, **Build**, **Flash**, **Terminal** and **Studio tools**. Studio tools gathers everything else: file switching, search across the workspace, output, debug controls, settings and the full command list.

Keyboard shortcuts

Keys	Action
`Ctrl` + `Alt` + `H`	Return to Workspace Home
`Ctrl` + `Alt` + `Space`	Open Studio tools
`Ctrl` + `P`	Open a file by name
`Ctrl` + `Tab`	Switch between open files
`Ctrl` + `S` / `Ctrl` + `W`	Save / close the current file
`Ctrl` + `Shift` + `B`	Build the active project
`Ctrl` + `Alt` + `F`	Flash (or deploy, on Linux boards)
`Ctrl` + `Alt` + `M`	Open the serial console (or run the app, on Linux boards)
`F5`	Start debugging (STM32 boards)

Buttons are only enabled when the active project is ready for them. If **Flash** is greyed out, Doctor will tell you what's missing.

> **Prefer the classic editor layout?**
> Turn off the `embedone.studioShell` setting. You get the familiar VS Code layout, with EmbedOne's panels in the sidebar: Workspace, Boards, Environment, Project and Devices.

4 Your first project: an STM32 Nucleo

This chapter walks through the whole cycle on a **NUCLEO-G071RB**. The same steps work for the NUCLEO-F401RE and NUCLEO-C5A3ZG. The starter project prints a greeting and then a counter over the serial port, so you can see that everything works.

Create the project

1. **Create a workspace**
   On Workspace Home choose **Create workspace**, give it a name, and pick a parent folder. EmbedOne makes a new folder for it and never overwrites an existing one.
2. **Add the board**
   Choose **MCU**, search for *G071*, select **NUCLEO-G071RB** and name the project. Its starter code is created beside the workspace file.
3. **Select the project card**
   This makes it the active project. Build, Flash, Monitor and Debug now act on it.

Set up the tools

1. **Check setup**
   This runs Doctor, which lists each tool the board needs and whether it's ready.
2. **Install dependencies**
   EmbedOne downloads the Arm GNU compiler, CMake, Ninja and the STM32G0 support files, verifies each against a pinned checksum, and installs them in its own folder. The first time takes a few minutes; later STM32 projects reuse them.
3. **Install STM32CubeCLT for flashing and debugging**
   See below. Building works without it.

> **STM32CubeCLT is a manual step**
> Flashing and debugging STM32 chips needs ST's programmer and GDB server, which come in **STM32CubeCLT**. ST requires a free myST account and asks you to accept its licence yourself, so EmbedOne can't download it for you.
>
> 1. Go to st.com/en/development-tools/stm32cubeclt.html and sign in.
> 2. Install the Windows x64 STM32CubeCLT, version 1.22.0 or newer, and follow ST's driver instructions.
> 3. Restart EmbedOne and run **Check setup** again.
> 4. If you installed it somewhere other than `C:\ST`, set the environment variable `EMBEDONE_CUBECLT_ROOT` to that folder before starting EmbedOne.

Build

Press `Ctrl`+`Shift`+`B`. When the build finishes, the project shows how much flash and RAM the firmware uses. If the compiler reports an error, it appears in the Problems list with the file and line.

Flash

Connect the Nucleo's USB connector (the one labelled ST-LINK) with a data cable. The board appears under **Devices**. Press `Ctrl`+`Alt`+`F`.

Before writing anything, EmbedOne reads the chip's device ID through the ST-LINK and compares it with the board your project targets. If you've plugged in a different board, it stops and writes nothing. After writing, it reads the flash back to confirm the firmware arrived intact, then resets the chip.

Monitor

Press `Ctrl`+`Alt`+`M`. A terminal opens on the board's serial port at 115200 baud and the starter firmware's output appears:

```
EmbedOne STM32G0 test
hello 0
hello 1
hello 2
```

Close the terminal or use **Stop Monitor** to release the port. Chapter 7 covers the console in more detail.

Debug

Press `F5`. EmbedOne builds the project if needed, starts the ST-LINK GDB server, loads the firmware and stops at the start of `main`. From there you can set breakpoints, step through code, inspect variables and browse the chip's peripheral registers. Stopping the debug session also stops the GDB server, so the probe is free for the next flash.

5 ESP32 and Raspberry Pi Pico

These boards are programmed as Arduino sketches. EmbedOne installs the Arduino command-line tool, the board's core and its compiler inside its own folder, pinned to tested versions. Your PC's own Arduino installation, if you have one, isn't touched.

ESP32 Dev Module

Create the project and run **Install dependencies**, then build. **Flash** uploads over the board's USB serial connection and **Monitor** opens the same port. EmbedOne detects the port automatically; if you have several boards connected, choose one with `--port COM7` from the command line.

Raspberry Pi Pico, Pico W, Pico 2, Pico 2 W

These use the community Arduino-Pico core. A build produces a `.uf2` file. For the first flash, **hold the BOOTSEL button while plugging in USB**; the Pico appears as a drive and EmbedOne copies the firmware onto it. If several Pico drives are connected, name one: `embedone flash --port "E:\"`.

Where the code goes

An Arduino sketch must live in a folder with the same name as its `.ino` file, so the starter project keeps it at `main/main.ino`. EmbedOne checks this before building, so a misplaced sketch gets a clear error.

Using libraries

List Arduino libraries in the project file with exact versions (chapter 10). Version ranges such as `^7.0.0` or `latest` are refused, so your build gives the same result next month as it does today.

> **Serial speed**
> An Arduino sketch chooses its own speed in `Serial.begin()`. The monitor uses the board's usual default; if your sketch uses another speed, pass it with `--baud`.

> **Why a Pico flash isn't marked "verified"**
> Copying a file to the BOOTSEL drive can't prove that the chip programmed itself correctly or started running. EmbedOne reports exactly what it knows rather than claiming more.

6 Linux boards: Raspberry Pi and NXP i.MX

On a Linux board, EmbedOne helps you write and run **applications**. It doesn't install or reflash the operating system: the board must already be running Linux and be reachable over the network.

Button	What it does on a Linux board
Build	Copies your source to the board over SSH and compiles it there with CMake and Ninja. The program comes back to your PC with a checksum.
Flash	Deploys the program into your account on the board, at `~/.local/share/embedone/apps/<project>/app`, and checks the copy. It doesn't touch the OS or reboot the board.
Monitor	Runs the program and shows its output. Anything you type goes to its input. **Stop** ends the program.

Before every operation EmbedOne checks the board's identity from its device tree, so a project for a Raspberry Pi 5 won't run against a Pi 4 by mistake.

Prepare the board

1. **Install the operating system and enable SSH**
   Raspberry Pi OS for a Raspberry Pi; a matching NXP Linux BSP for an i.MX board.

2. **Set up key-based SSH login**
   EmbedOne never types passwords. Create an SSH key on your PC and add it to your account on the board.

3. **Connect once yourself**
   Run `ssh your-user@raspberrypi.local` in a terminal, check the fingerprint, and accept it. EmbedOne won't accept an unknown board on your behalf.

4. **Install build tools on the board**
   On Raspberry Pi OS:

   ```
   sudo apt install python3 cmake ninja-build build-essential
   ```

   An i.MX image needs Python 3.8 or newer, CMake 3.16 or newer, Ninja and a C/C++ compiler. Minimal Yocto images may need these added when the image is built.

Tell EmbedOne where the board is

Open **Project settings**, or edit the project's `embedone.json`, and fill in the connection:

```
"settings": {
  "host": "raspberrypi.local",
  "user": "your-user",
  "port": 22
}
```

If your key isn't your default SSH key, add `"identity_file"` with the full path to the key file. Then run **Check setup**; Doctor confirms the connection, the board model and the build tools.

Limits to know

- Your CMake project must produce an executable named `app`, as the starter project does.
- Sources can be up to 16 MiB and 4,096 files; symbolic links aren't copied.
- Shared libraries your program needs must be installed on the board separately.
- The console is a simple text pipe, not a full terminal, so full-screen programs won't display properly.
- Debugging isn't available for Linux boards in 1.0.0.

7 The serial console

Monitor opens a terminal connected to the board. It's interactive: what you type is sent to the board, which is useful for firmware with a command prompt.

To	Do this
Open it	`Ctrl`+`Alt`+`M`, or **Monitor** on the project
Stop it and free the port	Close the terminal, or run **Stop Monitor**
See what you type	Turn on `embedone.monitor.localEcho` if your firmware doesn't echo characters back
Use a different port or speed	`embedone monitor --port COM7 --baud 9600`

On STM32 boards the speed comes from the board definition (115200). Only one program can use a serial port at a time, so close other serial tools before opening the console.

> **A burst of random characters when you first open it?**
> The first time you open the console after plugging in an STM32 Nucleo, it can start with a few lines of garbage before the real output. That's text the ST-LINK received before its serial speed was set. Close the console and open it again; it won't recur until you unplug the board.

8 Adding more boards

Beyond the 33 built-in boards, you can add boards from the official **Arduino** (AVR, megaAVR and SAMD) and **Espressif ESP32** catalogs. No account or API key is needed.

1. **Find the board**
   On Workspace Home, choose **Find another board** below the targets, pick a publisher, and search.
2. **Review and install**
   Select the catalog entry, review its version and checksum, and choose **Install and discover boards**.
3. **Select the exact board**
   EmbedOne compiles a small test program for it. Only a successful compile saves the board for future projects.

A compile proves the board's tools work; it doesn't prove that upload, pins or behaviour are right on your hardware. Added boards don't auto-detect their port, so pass `--port` when flashing and monitoring. Debugging isn't offered for them.

> **Optional: ask a local AI**
> **Ask local AI** can answer questions about a board using an AI model running on your own PC through Ollama. It only gives advice: it never installs anything, and its suggestions are never run. Nothing is sent to the internet.

9 The command line

The installer puts `embedone` on your PATH. Open a **new** PowerShell or Command Prompt window after installing, and run commands from inside a project folder, or point at one with `--path`. Command names aren't case-sensitive.

Command	What it does
`embedone doctor`	Check everything the project needs; `--install` also installs what's missing
`embedone install`	Install the project's dependencies, or a platform's: `embedone install stm32-g0`
`embedone build`	Build; `--clean` removes old output first
`embedone flash`	Program and verify the board (deploy, on Linux boards)
`embedone monitor`	Stream serial output until `Ctrl`+`C`; `--port`, `--baud`
`embedone devices`	List connected boards and serial ports
`embedone boards`	List supported boards
`embedone new <board>`	Create a starter project: `--path`, `--name`
`embedone catalog`	Search, prepare and add boards from catalogs (chapter 8)
`embedone debug`	Describe a debug session; press `F5` in the editor to debug
`embedone --help`	Full help; `embedone --version` prints the version

A complete session

```
embedone new nucleo-g071rb --path "C:\Projects\Blinky"
cd "C:\Projects\Blinky"
embedone doctor --install
embedone build
embedone flash
embedone monitor
```

For scripts and CI

Most commands accept `--json` and print one structured result, with `success` and, on failure, an `error` that says what went wrong. Scripts should rely on that rather than parsing the human-readable text.

10 The project file

Every project has an `embedone.json` at its root. EmbedOne finds it by looking in the current folder and its parents, so commands work from any subfolder.

```
{
  "name": "my-sensor",
  "modules": [
    { "id": "main", "type": "stm32", "board": "nucleo-g071rb", "source": "src" }
  ]
}
```

Field	Meaning
`name`	The project's display name
`type`	Which kind of target: `stm32`, `arduino` or `linux-app`
`board`	The board ID from appendix A
`source`	The source folder, relative to the project
`settings`	Optional, depends on the type: Arduino libraries, or a Linux board's SSH connection

Arduino libraries

```
"settings": {
  "libraries": [ { "name": "ArduinoJson", "version": "7.0.4" } ]
}
```

The board ID selects settings EmbedOne already knows, such as the memory layout or the compiler options, so a project file can never inject its own compiler flags or tool paths. An unknown field inside `settings` is reported as an error rather than silently ignored.

11 Files, folders and settings

What	Where
The application	`%LOCALAPPDATA%\Programs\EmbedOne`
Compilers, SDKs, board catalogs and downloads	`%LOCALAPPDATA%\EmbedOne`; change with the `embedone.home` setting or the `EMBEDONE_HOME` environment variable
ST's tools	Where you installed STM32CubeCLT, normally `C:\ST`; change with `EMBEDONE_CUBECLT_ROOT`
Your projects	Anywhere you choose. Don't put them inside the application folder

EmbedOne never changes the system-wide PATH or installs into Program Files. Deleting its folder removes everything it downloaded.

> **Keep the dependency folder's path short**
> Windows limits most file paths to 260 characters, and compilers build long paths of their own inside it. If you move the dependency folder, choose something short such as `D:\EmbedOne`.

Settings

Setting	What it changes
`embedone.home`	Where dependencies are installed. Empty uses the default
`embedone.cliPath`	Use a different `embedone` program. Empty uses the bundled one
`embedone.monitor.localEcho`	Show your typing in the serial console
`embedone.devices.refreshSeconds`	How often the Devices list checks for boards
`embedone.distributionWelcome`	Open Workspace Home at startup
`embedone.studioShell`	Use the compact EmbedOne layout. Turn off for the classic editor layout

12 Troubleshooting

Start with **Check setup**. Doctor checks each requirement separately and gives a specific fix for anything that isn't ready.

You see	Why	What to do
INSTALLING AND STARTING		
"Windows protected your PC"	The installer isn't code-signed yet	Check the SHA-256, then **More info** → **Run anyway**
A firewall prompt naming VSCodium	The editor base EmbedOne is built on	Choose **Cancel**; nothing needs incoming access
`embedone` isn't recognised in a terminal	The terminal was open before you installed	Open a new terminal window
STM32 BOARDS		
Build works but Flash and Debug are disabled	STM32CubeCLT wasn't found	Install it (chapter 4), restart EmbedOne; set `EMBEDONE_CUBECLT_ROOT` if it isn't in `C:\ST`
Devices says "No board detected"	The board is still starting up, the cable is charge-only, or the ST-LINK driver is missing	Wait a few seconds and refresh; use a data cable; install ST's driver
Flash refuses: device ID mismatch	A different chip is connected from the one the project targets	Connect the right board, or create a project for the one you have. Nothing was written
No flash/RAM figures after a build	Known issue when the compiler comes from STM32CubeCLT	The build itself is fine. A later release will fix this
A header "doesn't exist" but it's there	The path is longer than Windows allows	Move the dependency folder somewhere shorter (chapter 11)
SERIAL CONSOLE		
Random characters at the start	Output buffered before the speed was set	Close and reopen the console
Garbled text the whole time	The speed doesn't match your firmware	Use `--baud` with your firmware's speed
"Port is busy"	Another program has the port open	Close other serial tools and the other console
LINUX BOARDS		
"Host key verification failed"	You haven't connected to this board before	Run `ssh user@host` once, check the fingerprint, accept it
"Permission denied (publickey)"	Key login isn't set up	Add your public key on the board; set `identity_file` if needed

You see	Why	What to do
Doctor says the board's CMake, Ninja or compiler is missing	Build tools aren't installed on the board	Install them (chapter 6)
"Target model mismatch"	The project is for a different board model	Point the project at the right board, or create a project for this one
RASPBERRY PI PICO		
Flash can't find the Pico	It isn't in BOOTSEL mode	Hold BOOTSEL while plugging in USB
Flash asks which drive	More than one Pico drive is connected	`embedone flash --port "E:\"` with the right drive letter

13 Updating and uninstalling

Updating

EmbedOne doesn't update itself, so the version you installed keeps working exactly as tested. New releases appear on the EmbedOne website and at github.com/EmbedOne/embedone-web/releases. Run the new installer over the existing one; your projects and settings are kept.

Uninstalling

Open Windows **Settings** → **Apps**, find **EmbedOne** and choose **Uninstall**. This removes the application, its shortcuts, and the PATH entry the installer added.

To be safe, it leaves two things behind: your projects, and the downloaded compilers and SDKs in `%LOCALAPPDATA%\EmbedOne`. Delete that folder yourself if you want the disk space back. STM32CubeCLT is uninstalled separately, through ST's own uninstaller.

14 Known limitations in 1.0.0

- **Hardware testing.** The NUCLEO-G071RB has been tested end to end on real hardware: build, verified flash, serial console and debugging. The other boards have been checked in software and against their vendors' own files, but not yet on physical hardware.
- **Debugging** is available for the three STM32 boards only.
- **Linux boards** run applications; EmbedOne doesn't build or flash operating-system images, and doesn't debug Linux programs yet.
- **Pico flashing** can't be read back, so it isn't marked as verified.
- **STM32CubeCLT** has to be installed by hand, because ST requires an account.
- **Branding.** The installer isn't code-signed, and some Windows dialogs and file properties still say VSCodium.
- **Two known issues:** flash and RAM figures don't appear when the compiler comes from STM32CubeCLT, and the first serial session after plugging in a Nucleo can start with garbage (chapter 7).
- **Windows only.** A Linux version is coming.

A Supported boards

Use the board ID with `embedone new` or in `embedone.json`.

Board ID	Board	Chip	Works with
STM32 · TYPE `STM32`			
`nucleo-c5a3zg`	NUCLEO-C5A3ZG	STM32C5A3ZG	Build, flash, monitor, debug
`nucleo-f401re`	NUCLEO-F401RE	STM32F401RE	Build, flash, monitor, debug
`nucleo-g071rb`	NUCLEO-G071RB	STM32G071RB	Build, flash, monitor, debug
ARDUINO · TYPE `ARDUINO`			
`esp32-devkit`	ESP32 Dev Module	ESP32	Build, flash, monitor
`rpi-pico`	Raspberry Pi Pico	RP2040	Build, flash, monitor
`rpi-pico-w`	Raspberry Pi Pico W	RP2040	Build, flash, monitor
`rpi-pico-2`	Raspberry Pi Pico 2	RP2350	Build, flash, monitor
`rpi-pico-2-w`	Raspberry Pi Pico 2 W	RP2350	Build, flash, monitor
RASPBERRY PI LINUX · TYPE `LINUX-APP`			
`rpi-5`	Raspberry Pi 5	BCM2712	Build, deploy, run
`rpi-cm5`	Raspberry Pi Compute Module 5	BCM2712	Build, deploy, run
`rpi-4`	Raspberry Pi 4 Model B	BCM2711	Build, deploy, run
`rpi-400`	Raspberry Pi 400	BCM2711	Build, deploy, run
`rpi-cm4`	Raspberry Pi Compute Module 4	BCM2711	Build, deploy, run
`rpi-3b-plus`	Raspberry Pi 3 Model B+	BCM2837B0	Build, deploy, run
`rpi-3b`	Raspberry Pi 3 Model B	BCM2837	Build, deploy, run
`rpi-zero-2-w`	Raspberry Pi Zero 2 W	BCM2710A1	Build, deploy, run
`rpi-zero-w`	Raspberry Pi Zero W	BCM2835	Build, deploy, run
`rpi-zero`	Raspberry Pi Zero	BCM2835	Build, deploy, run
NXP I.MX LINUX · TYPE `LINUX-APP`			
`imx6q-sabresd`	i.MX 6Quad SABRE SD	i.MX 6Quad	Build, deploy, run
`imx6ul-14x14-evk`	i.MX 6UltraLite 14x14 EVK	i.MX 6UltraLite	Build, deploy, run
`imx6ull-14x14-evk`	i.MX 6ULL 14x14 EVK	i.MX 6ULL	Build, deploy, run
`imx7d-sdb`	i.MX 7Dual SABRE SD	i.MX 7Dual	Build, deploy, run

Board ID	Board	Chip	Works with
`imx7ulp-evk`	i.MX 7ULP EVK	i.MX 7ULP	Build, deploy, run
`imx8mm-evk`	i.MX 8M Mini EVK	i.MX 8M Mini	Build, deploy, run
`imx8mn-evk`	i.MX 8M Nano EVK	i.MX 8M Nano	Build, deploy, run
`imx8mp-evk`	i.MX 8M Plus EVK	i.MX 8M Plus	Build, deploy, run
`imx8mq-evk`	i.MX 8M Quad EVK	i.MX 8M Quad	Build, deploy, run
`imx8qm-mek`	i.MX 8QuadMax MEK	i.MX 8QuadMax	Build, deploy, run
`imx8qxp-mek`	i.MX 8QuadXPlus MEK	i.MX 8QuadXPlus	Build, deploy, run
`imx8ulp-evk`	i.MX 8ULP EVK	i.MX 8ULP	Build, deploy, run
`imx91-11x11-evk`	i.MX 91 11x11 EVK	i.MX 91	Build, deploy, run
`imx93-11x11-evk`	i.MX 93 11x11 EVK	i.MX 93	Build, deploy, run
`imx95-19x19-evk`	i.MX 95 19x19 EVK	i.MX 95	Build, deploy, run